

Parcial ASM 2025

Soluciones

Ejercicio 1

Traducir el siguiente código en C a ASM de la máquina virtual.

TEMA 0x <pre>typedef struct _nodo { short int id; void* dato; struct _nodo* sig; } nodo;</pre>	<pre>void insert_sort (nodo** h, nodo* n, int (*comp)(void*, void*)) { if ((*h == NULL) (comp(n->dato, (*h)->dato) <= 0)) { n->sig = *h; *h = n; } else { insert_sort(&((*h)->sig), n, comp); } }</pre>
TEMA 1x <pre>typedef struct _nodo { short int id; void* dato; struct _nodo* sig; } nodo;</pre>	<pre>void insert_sort (nodo** h, nodo* n, int (*comp)(void*, void*)) { register nodo** aux = h; while ((*aux != NULL) && (comp(n->dato, (*aux)->dato) > 0)) { aux = &((*aux)->sig); } n->sig = *aux; *aux = n; }</pre>

Ejercicio 1 (tema 0x - recursivo)

```
insert_sort:  push bp
              mov bp, sp
              push eax
              push ebx
              push ecx
              push edx

              mov edx, [bp+8]
              mov ecx, [bp+12]
```

```
insert_cond:  cmp [edx], null
              jz  insert_if
              mov ebx, [edx]
              push [ebx+dato]
              push [ecx+dato]
              call [bp+16]
              add sp, 8
              cmp eax, 0
              jnp insert_if
```

```
insert_else:  mov edx, [edx]
              add edx, sig
              push [bp+16]
              push ecx
              push edx
              call insert_sort
              add sp, 12
              jmp insert_fin
```

```
insert_if:    mov [ecx+sig], [edx]
              mov [edx], ecx
```

```
insert_fin:   pop edx
              pop ecx
              pop ebx
              pop eax
              mov sp, bp
              pop bp
              ret
```

```
; estructura nodo
id equ 0
dato equ 2
sig equ 6
null equ -1
```

```
;push <comp*>
;push <nodo*>
;push <head**>
;call insert_sort
;add sp, 12
```

```
;push <b>
;push <a>
;call comp
;add sp, 8
; retorna en eax
; > 0 si a > b
; = 0 si a = b
; < 0 si a < b
```

Ejercicio 1 (tema 0x - recursivo)

```
insert_sort:  push bp
              mov bp, sp
              push eax
              push ebx
              push ecx
              push edx
```

```
              mov edx, [bp+8]
              mov ecx, [bp+12]
```

```
insert_cond:  cmp [edx], null
              jz  insert_if
```

```
              mov ebx, [edx]
              push [ebx+dato]
              push [ecx+dato]
              call [bp+16]
              add sp, 8
```

```
              cmp eax, 0
              jnp insert_if
```

```
insert_else:  mov edx, [edx]
              add edx, sig
```

```
              push [bp+16]
              push ecx
              push edx
              call insert_sort
              add sp, 12
              jmp insert_fin
```

```
insert_if:    mov [ecx+sig], [edx]
              mov [edx], ecx
```

```
insert_fin:   pop edx
              pop ecx
              pop ebx
              pop eax
              mov sp, bp
              pop bp
              ret
```

```
; estructura nodo
id equ 0
dato equ 2
sig equ 6
null equ -1
```

```
;push <comp*>
;push <nodo*>
;push <head**>
;call insert_sort
;add sp, 12
```

```
;push <b>
;push <a>
;call comp
;add sp, 8
; retorna en eax
; > 0 si a > b
; = 0 si a = b
; < 0 si a < b
```

Ejercicio 1 (tema 1x - iterativo)

```
insert_sort:  push bp
              mov bp, sp
              push eax
              push ebx
              push ecx
              push edx

              mov edx, [bp+8]
              mov ecx, [bp+12]

insert_sigue: cmp [edx], null
              jz  insert_fin
              mov ebx, [edx]
              push [ebx+dato]
              push [ecx+dato]
              call [bp+16]
              add sp, 8
              cmp eax, 0
              jnp insert_fin

insert_ciclo: mov edx, [edx]
              add edx, sig
              jmp insert_sigue

insert_fin:  mov [ecx+sig], [edx]
              mov [edx], ecx

              pop edx
              pop ecx
              pop ebx
              pop eax
              mov sp, bp
              pop bp
              ret
```

```
; estructura nodo
id equ 0
dato equ 2
sig equ 6
null equ -1
```

```
;push <comp*>
;push <nodo*>
;push <head**>
;call insert_sort
;add sp, 12
```

```
;push <b>
;push <a>
;call comp
;add sp, 8
; retorna en eax
; > 0 si a > b
; = 0 si a = b
; < 0 si a < b
```

Ejercicio 1 (tema 1x - iterativo)

```
insert_sort:  push bp
              mov bp, sp
              push eax
              push ebx
              push ecx
              push edx
```

```
              mov edx, [bp+8]
              mov ecx, [bp+12]
```

```
insert_sigue: cmp [edx], null
              jz  insert_fin
```

```
              mov ebx, [edx]
              push [ebx+dato]
              push [ecx+dato]
              call [bp+16]
              add sp, 8
```

```
              cmp eax, 0
              jnp insert_fin
```

```
insert_ciclo: mov edx, [edx]
              add edx, sig
```

```
              jmp insert_sigue
```

```
insert_fin:  mov [ecx+sig], [edx]
              mov [edx], ecx
```

```
              pop edx
              pop ecx
              pop ebx
              pop eax
              mov sp, bp
              pop bp
              ret
```

```
; estructura nodo
id equ 0
dato equ 2
sig equ 6
null equ -1
```

```
;push <comp*>
;push <nodo*>
;push <head**>
;call insert_sort
;add sp, 12
```

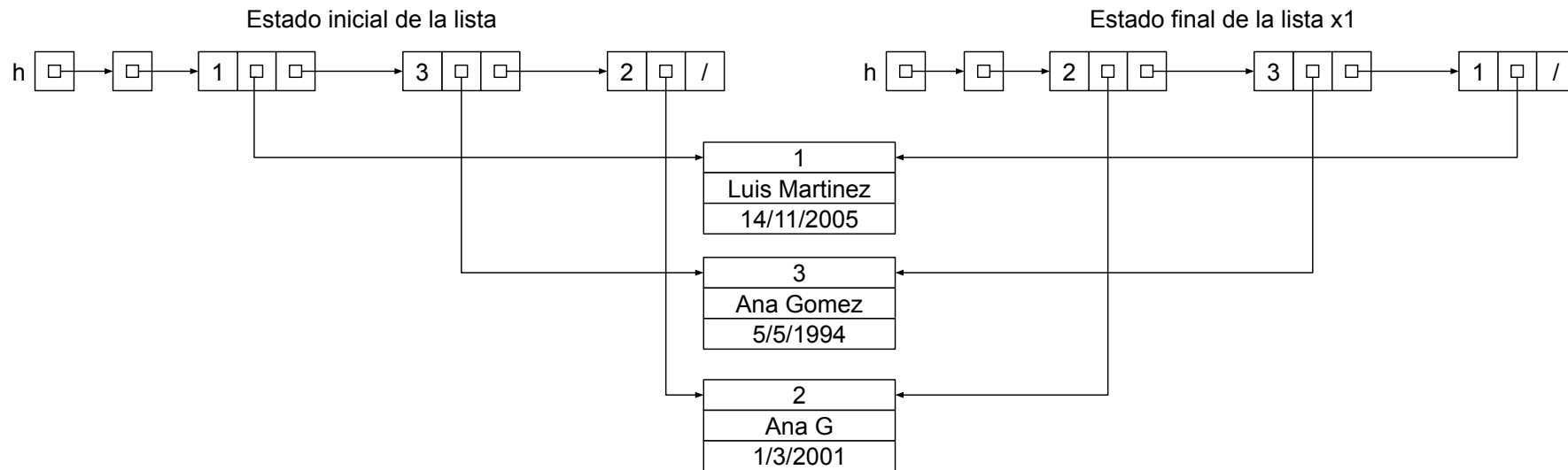
```
;push <b>
;push <a>
;call comp
;add sp, 8
; retorna en eax
; > 0 si a > b
; = 0 si a = b
; < 0 si a < b
```

Ejercicio 2

Utilizando la subrutina del ejercicio 1, desarrollar la subrutina “**SORT_LIST**”, y todas las necesarias, para que dada una lista no ordenada de contactos, **ordene la lista según el XXX del contacto**.

TEMA x0: Nombre

TEMA x1: Cumpleaños



Ejercicio 2 - cmp_contacto (tema x0 - nombre)

```
cmp_contacto: push bp
               mov bp, sp
               push ebx

               mov eax, [bp+8]
               mov ebx, [bp+12]

               add eax, nombre
               add ebx, nombre

otro_char:     cmp b[eax], 0
               jz  cmp_str_fin
               cmp b[ebx], 0
               jz  cmp_str_fin
               cmp b[eax], b[ebx]
               jnz cmp_str_fin

               add eax, 1
               add ebx, 1

               jmp otro_char

cmp_str_fin:   mov eax, b[eax]
               sub eax, b[ebx]
               pop ebx
               mov sp, bp
               pop bp
               ret
```

```
; estructura contacto
id equ 0
nombre equ 2
fecha equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia
```


Ejercicio 2 - cmp_contacto (tema x1 - cumpleaños)

```
cmp_contacto: push bp
               mov bp, sp
               push ebx

               mov eax, [bp+8]
               mov ebx, [bp+12]

               add eax, w[eax+fecha]
               add ebx, w[ebx+fecha]

               and eax, 0x01FF
               and ebx, 0x01FF

cmp_str_fin:   sub eax, ebx
               pop ebx
               mov sp, bp
               pop bp
               ret
```

```
; estructura contacto
id equ 0
nombre equ 2
fecha equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia
```

Ejercicio 2 - sort_list (1) - Iterativo sin lista auxiliar

```
sort_list:    push bp
              mov bp, sp
              push eax
              push ebx
              push edx

              mov edx, [bp+8]
              mov ebx, [edx]
              mov [edx], null

otro_nodo:    cmp ebx, null
              jz sort_list_fin

              mov eax, ebx
              mov ebx, [ebx+sig]

              push cmp_contacto
              push eax
              push edx
              call insert_sort
              add sp, 12

              jmp otro_nodo

sort_list_fin: pop edx
               pop ebx
               pop eax
               mov sp, bp
               pop bp
               ret

; estructura contacto
id equ 0
nombre equ 2
fecha equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia

; estructura lista (sin id)
dato equ 2
sig equ 6

null equ -1

;push <nodo** h>
;call sort_list
;add sp, 8
;no retorna nada
```

Ejercicio 2 - sort_list (1) - Iterativo sin lista auxiliar

```
sort_list:    push bp
              mov bp, sp
              push eax
              push ebx
              push edx

              mov edx, [bp+8]
              mov ebx, [edx]
              mov [edx], null

otro_nodo:    cmp ebx, null
              jz sort_list_fin

              mov eax, ebx
              mov ebx, [ebx+sig]

              push cmp_contacto
              push eax
              push edx
              call insert_sort
              add sp, 12

              jmp otro_nodo

sort_list_fin: pop edx
               pop ebx
               pop eax
               mov sp, bp
               pop bp
               ret
```

```
...
otro_nodo:    cmp ebx, null
              jz sort_list_fin

              mov eax, ebx
              mov ebx, [ebx+sig]

              push cmp_contacto
              push eax
              push edx
              call insert_sort
              add sp, 12

              jmp otro_nodo

...
```

```
; estructura contacto
id equ 0
nombre equ 2
fecha equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia

; estructura lista (sin id)
dato equ 2
sig equ 6

null equ -1

;push <nodo** h>
;call sort_list
;add sp, 8
;no retorna nada
```

Ejercicio 2 - sort_list (2) - Iterativo con lista auxiliar

```
sort_list:    push bp
              mov bp, sp
              sub sp, 4
              push ebx
              push edx
              push efx

              mov efx, bp
              sub efx, 4 ;lista aux
              mov [efx], null

              mov edx, [bp+8]
              mov ebx, [edx]
```

```
otro_nodo:    cmp ebx, null
              jz sort_list_fin

              mov [edx], [ebx+sig]

              push cmp_contacto
              push ebx
              push efx
              call insert_sort
              add sp, 12

              mov ebx, [edx]

              jmp otro_nodo
```

```
sort_list_fin: mov [edx], [efx]
                pop efx
                pop edx
                pop ebx
                mov sp, bp
                pop bp
                ret
```

```
; estructura contacto
id equ 0
nombre equ 2
fecha equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia
```

```
; estructura lista (sin id)
dato equ 2
sig equ 6
```

```
null equ -1

;push <nodo** h>
;call sort_list
;add sp, 8
;no retorna nada
```

Ejercicio 2 - sort_list (3) - Recursivo con lista auxiliar

```
sort_list:    push bp
              mov bp, sp
              sub sp, 4
              push ebx
              push edx
              push efx
              mov efx, bp
              sub efx, 4 ;lista aux
              mov [efx], null

              mov edx, [bp+8]
              mov ebx, [edx]

              push ebx
              push efx
              call _sort_list
              add sp, 12

              mov [edx], [efx]
```

```
sort_list_fin: pop efx
               pop edx
               pop ebx
               mov sp, bp
               pop bp
               ret
```

```
_sort_list:   push bp
              mov bp, sp
              push ebx

              mov ebx, [bp+8] ;origen

              cmp ebx, null
              jz sort_list_fin

              push [ebx+sig]
              push [bp+12]
              call _sort_list
              add sp, 12

              push cmp_contacto
              push ebx
              push [bp+12] ;destino
              call insert_sort
              add sp, 12
```

```
sort_list_fin: pop ebx
               mov sp, bp
               pop bp
               ret
```

```
; estructura contacto
id equ 0
nombre equ 2
tel equ 52

;push <void* b>
;push <void* a>
;call cmp_contactos
;add sp, 8
; retorna en eax la diferencia

; estructura lista (sin id)
dato equ 2
sig equ 6

null equ -1

;push <nodo** h>
;call sort_list
;add sp, 8
;no retorna nada

;push <nodo* origen>
;push <nodo** destino>
;call _sort_list
;add sp, 12
;no retorna nada
```

Ejercicio 2 - sort_list (4) - Recursivo sin lista auxiliar

```
sort_list:      push bp
                mov bp, sp
                push ebx
                push edx

                mov edx, [bp+8]
                mov ebx, [edx]

                mov [edx], null

                push ebx
                push edx
                call _sort_list
                add sp, 12

sort_list_fin:  pop ebx
                pop edx
                pop ebx
                mov sp, bp
                pop bp
                ret

_sort_list:     push bp
                mov bp, sp
                push ebx

                mov ebx, [bp+8] ;origen

                cmp ebx, null
                jz sort_list_fin

                push [ebx+sig]
                push [bp+12]
                call _sort_list
                add sp, 12

                push cmp_contacto
                push ebx
                push [bp+12] ;destino
                call insert_sort
                add sp, 12

sort_list_fin: pop ebx
                mov sp, bp
                pop bp
                ret

                ; estructura contacto
                id equ 0
                nombre equ 2
                tel equ 52

                ;push <void* b>
                ;push <void* a>
                ;call cmp_contactos
                ;add sp, 8
                ; retorna en eax la diferencia

                ; estructura lista (sin id)
                dato equ 2
                sig equ 6

                null equ -1

                ;push <nodo** h>
                ;call sort_list
                ;add sp, 8
                ;no retorna nada

                ;push <nodo* origen>
                ;push <nodo** destino>
                ;call _sort_list
                ;add sp, 12
                ;no retorna nada
```